JAVA 8 TO JAVA 21

The visual, interactive, code-first journey

How to use this workbook

Read ONE small lesson. Before looking at the new code, predict the problem. Trace the arrows. Run both versions. Answer the question on paper. Only then check the answer on the next page.



No assumed jargon

Words like caller (the code that invokes a method), mutable (can change after creation), snapshot (a picture of data at one moment), and reference (a value that identifies an object) are explained when they first appear. Basic Java syntax is the only prerequisite.

What is included

A guided lesson for every version from Java 9 to Java 21, plus extra lessons for collections, records, pattern matching, and virtual threads. Preview features are clearly labeled; a preview is an experimental language/API feature that may change before becoming final.

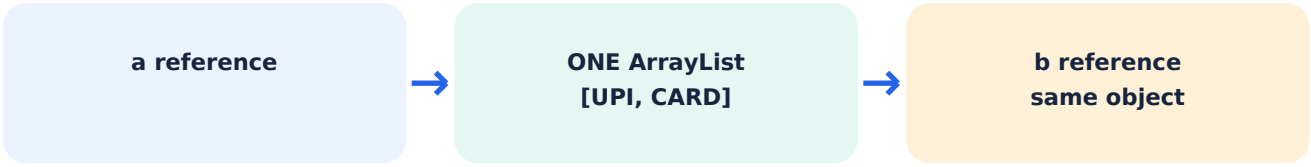
Important learning rule

A new Java version contains earlier released features. You can run most of these examples using JDK 21; preview examples may require special flags or may not be available in their old preview form.

Start here: objects, references and copies

Before Java 9, we need one memory idea. A reference is how a variable finds an object. Two variables can refer to the same object.

```
List<String> a = new ArrayList<>();
a.add("UPI");
List<String> b = a;
b.add("CARD");
System.out.println(a); // [UPI, CARD]
```



A shallow copy creates a new list but does not duplicate the objects stored inside it. If those objects can change, both lists may observe the change.

```
List<Payment> a = new ArrayList<>();
a.add(new Payment("UPI"));
List<Payment> b = new ArrayList<>(a);
b.get(0).method = "CARD";
System.out.println(a.get(0).method); // CARD
```

Why this matters

The list containers are separate. Their first elements still point to the same Payment object. Replacing an element in b would not replace the element in a.

Pause & predict

If `b.add(new Payment("CASH"))` runs, does `a.size()` change? Explain why before turning the page.

Answer to the previous page

No. a and b are different list containers, so adding to b does not add to a. Only the existing Payment element object is shared.

Java 9 | Unmodifiable lists: List.of()

FIRST: what problem are we solving?

You want a fixed set of allowed payment methods. Other code must not add a new method.

Before this feature: what developers wrote

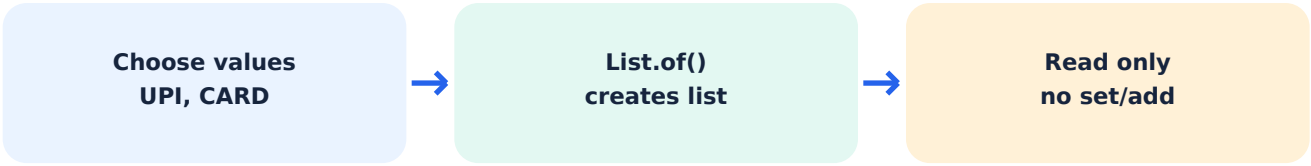
```
List<String> x = Collections.unmodifiableList(
    Arrays.asList("UPI", "CARD"));
```

What is annoying about the old approach?

You want a fixed set of allowed payment methods. Other code must not add a new method. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
List<String> x = List.of("UPI", "CARD");
```



What improves	What to watch out for
Shorter code; add, remove and set are blocked.	No null values. Mutable objects inside can still change.

Pause & predict - do not peek

Does x.set(0, "CASH") work?

Key point to remember

Unmodifiable lists: List.of(): Shorter code; add, remove and set are blocked.

Check your reasoning | Java 9

Answer

No. It throws UnsupportedOperationException.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 9. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 9 | Set.of() and Map.of()

FIRST: what problem are we solving?

You need a small set of unique roles or a small lookup table.

Before this feature: what developers wrote

```
Set<String> roles = new HashSet<>(
    Arrays.asList("ADMIN", "USER"));
```

What is annoying about the old approach?

You need a small set of unique roles or a small lookup table. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
Set<String> roles = Set.of("ADMIN", "USER");
Map<String,Integer> limits = Map.of("UPI", 10, "CARD", 5);
```



What improves	What to watch out for
Easy to make small unmodifiable sets and maps.	Set.of rejects duplicate elements; Map.of rejects duplicate keys; both reject null.

Pause & predict - do not peek

What happens with Set.of("A", "A")?

Key point to remember

Set.of() and Map.of(): Easy to make small unmodifiable sets and maps.

Check your reasoning | Java 9

Answer

IllegalArgumentException: duplicates are rejected.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 9. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 9 | Stream: takeWhile and dropWhile

FIRST: what problem are we solving?

You read ordered events until the first failure; filter() does not mean stop at the first failure.

Before this feature: what developers wrote

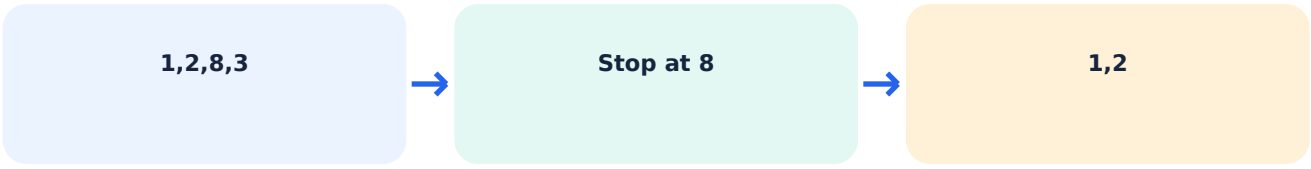
```
list.stream().filter(x -> x < 5)
    .forEach(System.out::println);
```

What is annoying about the old approach?

You read ordered events until the first failure; filter() does not mean stop at the first failure. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
list.stream().takeWhile(x -> x < 5)
    .forEach(System.out::println);
```



What improves	What to watch out for
takeWhile stops at the first mismatch; dropWhile skips the starting matching run.	Results depend on encounter order; an unordered stream can behave differently.

Pause & predict - do not peek

For [1, 2, 8, 3], what does takeWhile(x < 5) return?

Key point to remember

Stream: takeWhile and dropWhile: takeWhile stops at the first mismatch; dropWhile skips the starting matching run.

Check your reasoning | Java 9

Answer

[1, 2]. It stops when it meets 8.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 9. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 9 | Optional and interfaces

FIRST: what problem are we solving?

An Optional might be empty; interfaces may repeat helper code.

Before this feature: what developers wrote

```
if (opt.isPresent()) use(opt.get());
else use("default");
```

What is annoying about the old approach?

An Optional might be empty; interfaces may repeat helper code. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
opt.ifPresentOrElse(
    x -> use(x), () -> use("default"));
```



What improves	What to watch out for
Express both branches directly; private interface methods can share internal logic.	Optional is not a replacement for every nullable field.

Pause & predict - do not peek

If opt is empty, which branch runs?

Key point to remember

Optional and interfaces: Express both branches directly; private interface methods can share internal logic.

Check your reasoning | Java 9

Answer

The second Runnable: use("default").

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 9. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 9 | Modules and JShell

FIRST: what problem are we solving?

Large apps need clearer dependencies; tiny experiments should not require a whole project.

Before this feature: what developers wrote

```
// Older apps mainly used the classpath.
// Compile a class, then run it.
```

What is annoying about the old approach?

Large apps need clearer dependencies; tiny experiments should not require a whole project. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// module-info.java
module payments {
    exports com.example.payments;
}
// Terminal: jshell
```



What improves	What to watch out for
Modules declare what a module exposes. JShell runs small Java snippets interactively.	Modules add setup and migration complexity; not every application needs them.

Pause & predict - do not peek

Does exports make every package public?

Key point to remember

Modules and JShell: Modules declare what a module exposes. JShell runs small Java snippets interactively.

Check your reasoning | Java 9

Answer

No. Only the named package is exported.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 9. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 10 | var: local variable type inference

FIRST: what problem are we solving?

Repeating a long type name makes local code harder to scan.

Before this feature: what developers wrote

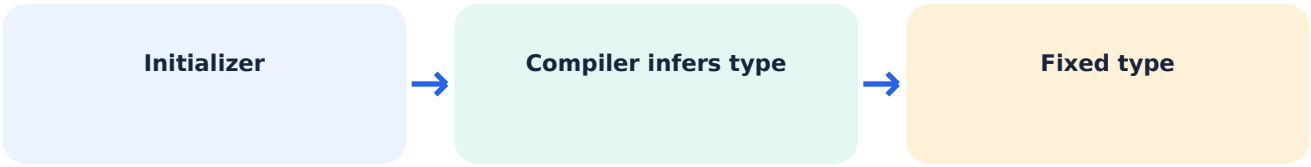
```
Map<String, List<Integer>> data =
    new HashMap<String, List<Integer>>();
```

What is annoying about the old approach?

Repeating a long type name makes local code harder to scan. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
var data = new HashMap<String, List<Integer>>();
```



What improves	What to watch out for
The compiler infers the local variable type from the initializer.	var is not dynamic typing; cannot use it for fields or without an initializer.

Pause & predict - do not peek

Can you write var x = null; ?

Key point to remember

var: local variable type inference: The compiler infers the local variable type from the initializer.

Check your reasoning | Java 10

Answer

No. Java cannot infer a type from null alone.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 10. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 10 | Unmodifiable collectors

FIRST: what problem are we solving?

You want a stream result that callers cannot edit.

Before this feature: what developers wrote

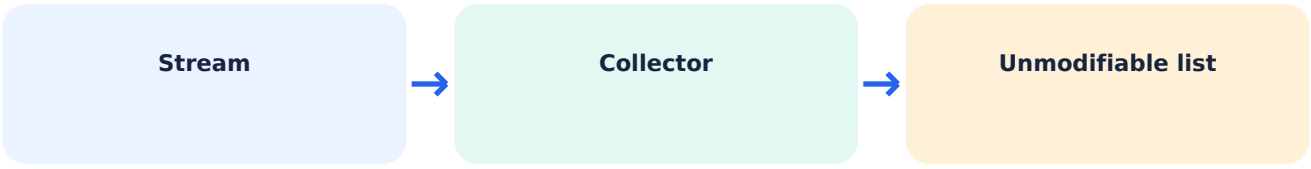
```
List<String> out = Collections.unmodifiableList(
    names.stream().collect(Collectors.toList()));
```

What is annoying about the old approach?

You want a stream result that callers cannot edit. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
List<String> out = names.stream().collect(
    Collectors.toUnmodifiableList());
```



What improves	What to watch out for
Build an unmodifiable result directly.	Rejects null elements. Collectors.toList() has different guarantees.

Pause & predict - do not peek

Can you call out.add("X")?

Key point to remember

Unmodifiable collectors: Build an unmodifiable result directly.

Check your reasoning | Java 10

Answer

No: UnsupportedOperationException.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 10. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 11 | String and file helpers

FIRST: what problem are we solving?

Checking blank text and reading a short file takes repetitive code.

Before this feature: what developers wrote

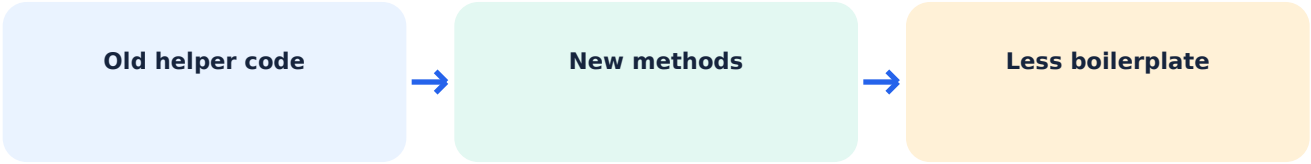
```
boolean blank = s.trim().isEmpty();
String txt = new String(Files.readAllBytes(path),
    StandardCharsets.UTF_8);
```

What is annoying about the old approach?

Checking blank text and reading a short file takes repetitive code. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
boolean blank = s.isBlank();
String txt = Files.readString(path);
```



What improves	What to watch out for
isBlank recognizes Unicode whitespace; Files.readString is shorter.	strip() and trim() differ; readString loads the whole file into memory.

Pause & predict - do not peek

Is " ".isBlank() true?

Key point to remember

String and file helpers: isBlank recognizes Unicode whitespace; Files.readString is shorter.

Check your reasoning | Java 11

Answer

Yes. All characters are whitespace.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 11. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 11 | HTTP Client

FIRST: what problem are we solving?

Sending web requests often needs verbose connection code or another library.

Before this feature: what developers wrote

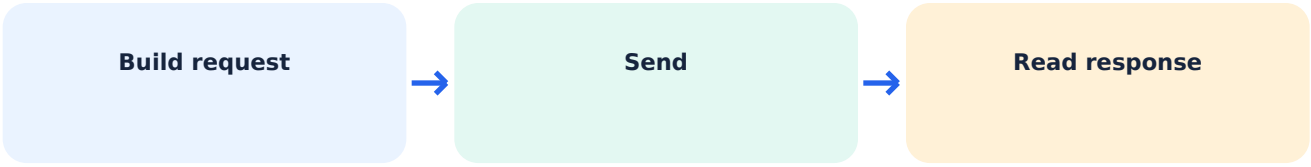
```
HttpURLConnection c = (HttpURLConnection)
    url.openConnection();
```

What is annoying about the old approach?

Sending web requests often needs verbose connection code or another library. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(uri).build();
HttpResponse<String> r = client.send(req,
    HttpResponse.BodyHandlers.ofString());
```



What improves	What to watch out for
Modern request/response API with HTTP/2 and asynchronous support.	Network calls can fail; set timeouts and handle errors.

Pause & predict - do not peek

Does client.send block the calling thread?

Key point to remember

HTTP Client: Modern request/response API with HTTP/2 and asynchronous support.

Check your reasoning | Java 11

Answer

Yes. sendAsync is the asynchronous alternative.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 11. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 12 | Switch expressions: preview

FIRST: what problem are we solving?

A switch that computes a value needs a variable and many breaks.

Before this feature: what developers wrote

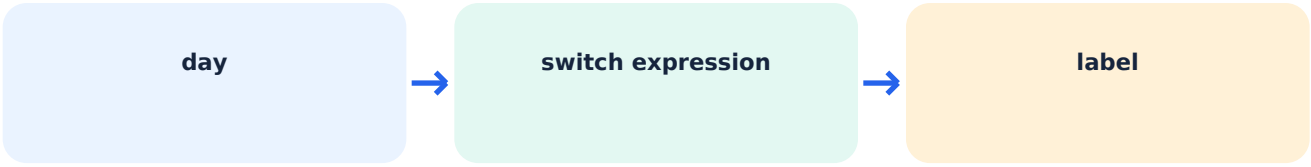
```
String label;
switch (day) {
 case 1: label = "Mon"; break;
 default: label = "Other";
}
```

What is annoying about the old approach?

A switch that computes a value needs a variable and many breaks. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in Java 12; finalized in Java 14
String label = switch (day) {
 case 1 -> "Mon";
 default -> "Other";
};
```



What improves	What to watch out for
A switch can return a value directly, with fewer accidental fall-through bugs.	Java 12 required preview flags; learn the final Java 14 form.

Pause & predict - do not peek

What happens without default if cases are not exhaustive?

Key point to remember

Switch expressions: preview: A switch can return a value directly, with fewer accidental fall-through bugs.

Check your reasoning | Java 12

Answer

A switch expression must cover all possible values.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 12. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 13 | Text blocks: preview

FIRST: what problem are we solving?

Multiline JSON in a Java string is hard to read because of escapes.

Before this feature: what developers wrote

```
String json = "{
  \"ok\": true
}";
```

What is annoying about the old approach?

Multiline JSON in a Java string is hard to read because of escapes. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in 13; finalized in 15
String json = """
    { "ok": true }
    """;
```



What improves	What to watch out for
Multiline strings look closer to their actual content.	Indentation and trailing newline behavior matter.

Pause & predict - do not peek

Are text blocks a different String type?

Key point to remember

Text blocks: preview: Multiline strings look closer to their actual content.

Check your reasoning | Java 13

Answer

No. They are ordinary java.lang.String values.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 13. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 14 | Switch expressions: final

FIRST: what problem are we solving?

You want to compute a result from a switch without repeated assignment.

Before this feature: what developers wrote

```
int n;
switch (grade) {
 case "A": n = 5; break;
 default: n = 0;
}
```

What is annoying about the old approach?

You want to compute a result from a switch without repeated assignment. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
int n = switch (grade) {
 case "A" -> 5;
 default -> 0;
};
```



What improves	What to watch out for
Clearer value-producing control flow.	Use yield for multi-statement switch-expression branches.

Pause & predict - do not peek

Does -> fall through into the next case?

Key point to remember

Switch expressions: final: Clearer value-producing control flow.

Check your reasoning | Java 14

Answer

No. Arrow cases do not fall through.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 14. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 14 | Helpful NullPointerExceptions

FIRST: what problem are we solving?

A chain like a.b.c() fails, but the old error often hid which part was null.

Before this feature: what developers wrote

```
// Old NPE messages often gave little detail.
order.customer.address.city();
```

What is annoying about the old approach?

A chain like a.b.c() fails, but the old error often hid which part was null. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Same code; newer JVM error often says
// which variable/field was null.
order.customer.address.city();
```



What improves	What to watch out for
Better error messages speed up debugging.	Message details depend on JVM settings and context; prevent nulls where possible.

Pause & predict - do not peek

Does this feature stop NullPointerException?

Key point to remember

Helpful NullPointerExceptions: Better error messages speed up debugging.

Check your reasoning | Java 14

Answer

No. It only helps explain the failure.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 14. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 15 | Text blocks: final

FIRST: what problem are we solving?

Writing SQL or JSON with lots of \n and escaped quotes is noisy.

Before this feature: what developers wrote

```
String sql = "SELECT id
" +
             "FROM payments";
```

What is annoying about the old approach?

Writing SQL or JSON with lots of \n and escaped quotes is noisy. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
String sql = """
    SELECT id
    FROM payments
    """;
```



What improves	What to watch out for
Readable multiline text in ordinary String variables.	Whitespace is significant; test exact output when formatting matters.

Pause & predict - do not peek

Can you call sql.length()?

Key point to remember

Text blocks: final: Readable multiline text in ordinary String variables.

Check your reasoning | Java 15

Answer

Yes. A text block is a String.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 15. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 15 | Sealed classes: preview

FIRST: what problem are we solving?

A payment type should allow only specific implementations.

Before this feature: what developers wrote

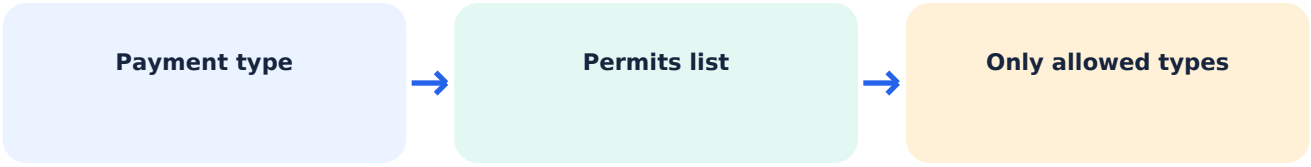
```
interface Payment { }
// Any class could implement Payment.
```

What is annoying about the old approach?

A payment type should allow only specific implementations. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in 15; final in 17
sealed interface Payment permits Card, Upi {}
final class Card implements Payment {}
final class Upi implements Payment {}
```



What improves	What to watch out for
Restrict who may extend/implement a type.	Permitted implementations must follow final, sealed or non-sealed rules.

Pause & predict - do not peek

Can an unrelated Cash class implement Payment?

Key point to remember

Sealed classes: preview: Restrict who may extend/implement a type.

Check your reasoning | Java 15

Answer

No, unless permitted.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 15. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 16 | Records: final

FIRST: what problem are we solving?

A small data carrier needs constructor, accessors, equals and toString.

Before this feature: what developers wrote

```
class User {
 final String name;
 User(String n) { name = n; }
 String name() { return name; }
}
```

What is annoying about the old approach?

A small data carrier needs constructor, accessors, equals and toString. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
record User(String name) {}
User u = new User("Ana");
System.out.println(u.name());
```



What improves	What to watch out for
Less boilerplate for data-focused classes.	Records are shallowly immutable; a mutable field object can still change.

Pause & predict - do not peek

Can a record extend another class?

Key point to remember

Records: final: Less boilerplate for data-focused classes.

Check your reasoning | Java 16

Answer

No. Records already extend java.lang.Record.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 16. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 16 | instanceof pattern matching: final

FIRST: what problem are we solving?

You test an object type and then cast it manually.

Before this feature: what developers wrote

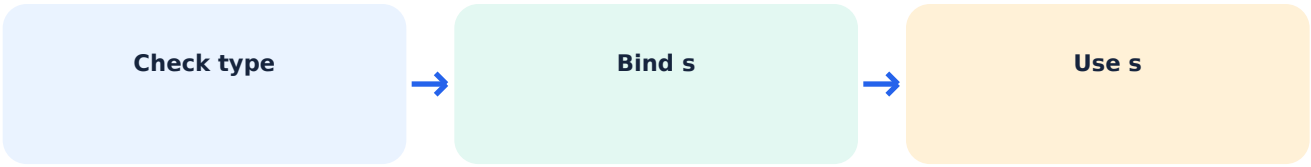
```
if (value instanceof String) {
 String s = (String) value;
 System.out.println(s.length());
}
```

What is annoying about the old approach?

You test an object type and then cast it manually. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
if (value instanceof String s) {
 System.out.println(s.length());
}
```



What improves	What to watch out for
Combines type check and a safely typed local variable.	Pattern variable scope depends on control flow.

Pause & predict - do not peek

If value is null, does the body run?

Key point to remember

instanceof pattern matching: final: Combines type check and a safely typed local variable.

Check your reasoning | Java 16

Answer

No. null does not match String.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 16. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 16 | Stream.toList()

FIRST: what problem are we solving?

Collecting a stream to a list is common and verbose.

Before this feature: what developers wrote

```
List<String> x = stream.collect(
    Collectors.toList());
```

What is annoying about the old approach?

Collecting a stream to a list is common and verbose. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
List<String> x = stream.toList();
```



What improves	What to watch out for
Shorter terminal operation returning an unmodifiable list.	Do not assume it behaves like mutable Collectors.toList(); null handling also differs from toUnmodifiableList().

Pause & predict - do not peek

Can x.add("new") work?

Key point to remember

Stream.toList(): Shorter terminal operation returning an unmodifiable list.

Check your reasoning | Java 16

Answer

No. Stream.toList() returns an unmodifiable list.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 16. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 17 | Sealed classes: final

FIRST: what problem are we solving?

Your model should allow a controlled family of result types.

Before this feature: what developers wrote

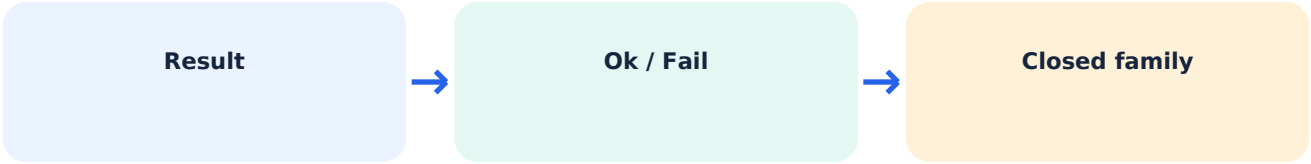
```
interface Result {}
// Any implementation is possible.
```

What is annoying about the old approach?

Your model should allow a controlled family of result types. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
sealed interface Result permits Ok, Fail {}
record Ok(String value) implements Result {}
record Fail(String reason) implements Result {}
```



What improves	What to watch out for
Models a known set of alternatives, useful with pattern matching.	Permitted types must be accessible and obey inheritance rules.

Pause & predict - do not peek

Can you add a new implementation without editing permits?

Key point to remember

Sealed classes: final: Models a known set of alternatives, useful with pattern matching.

Check your reasoning | Java 17

Answer

Not unless it is permitted under the declaration rules.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 17. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 17 | Strong JDK encapsulation

FIRST: what problem are we solving?

Old code sometimes accessed internal JDK classes not meant for applications.

Before this feature: what developers wrote

```
// Legacy code might import sun.* APIs.
// These were never stable public contracts.
```

What is annoying about the old approach?

Old code sometimes accessed internal JDK classes not meant for applications. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Prefer documented java.* APIs.
// Upgrade libraries that depend on internals.
```



What improves	What to watch out for
More reliable boundaries inside the JDK.	Older reflection-heavy libraries may need updates or migration flags.

Pause & predict - do not peek

Should you rely on sun.misc internal APIs?

Key point to remember

Strong JDK encapsulation: More reliable boundaries inside the JDK.

Check your reasoning | Java 17

Answer

Avoid undocumented internal APIs where possible.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 17. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 18 | UTF-8 default charset

FIRST: what problem are we solving?

The same text file can behave differently on machines with different default encodings.

Before this feature: what developers wrote

```
// Older defaults depended on OS / locale.
new FileReader("data.txt");
```

What is annoying about the old approach?

The same text file can behave differently on machines with different default encodings. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Java 18 defaults to UTF-8 for standard APIs.
Files.readString(Path.of("data.txt"));
```



What improves	What to watch out for
More consistent text behavior across systems.	Specify a charset explicitly when a file uses a different encoding.

Pause & predict - do not peek

Will a legacy non-UTF-8 file always decode correctly?

Key point to remember

UTF-8 default charset: More consistent text behavior across systems.

Check your reasoning | Java 18

Answer

No. Specify its actual charset.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 18. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 19 | Virtual threads: preview

FIRST: what problem are we solving?

A server waits on many network calls; one expensive platform thread per task can limit scale.

Before this feature: what developers wrote

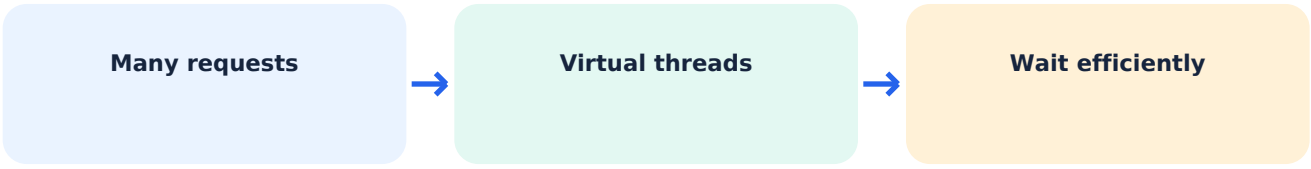
```
ExecutorService pool =
    Executors.newFixedThreadPool(100);
```

What is annoying about the old approach?

A server waits on many network calls; one expensive platform thread per task can limit scale. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in 19/20; final in 21
Thread.startVirtualThread(() -> fetchData());
```



What improves	What to watch out for
Cheap threads make blocking-style I/O concurrency easier to scale.	Not faster for CPU-heavy work; preview in 19.

Pause & predict - do not peek

Do virtual threads make calculations faster?

Key point to remember

Virtual threads: preview: Cheap threads make blocking-style I/O concurrency easier to scale.

Check your reasoning | Java 19

Answer

No. They mainly help high-concurrency blocking I/O.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 19. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 19 | Record patterns: preview

FIRST: what problem are we solving?

You receive a record and repeatedly call accessors to unpack it.

Before this feature: what developers wrote

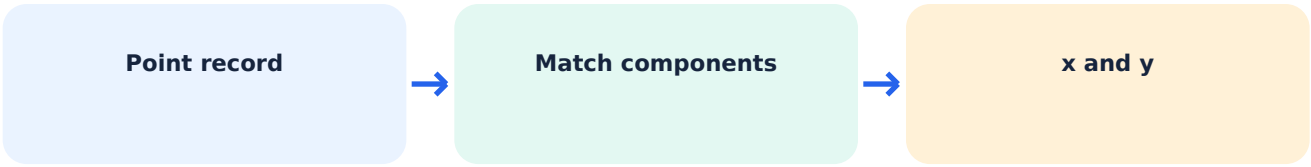
```
if (o instanceof Point p) {
 System.out.println(p.x());
}
```

What is annoying about the old approach?

You receive a record and repeatedly call accessors to unpack it. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in 19/20; final in 21
if (o instanceof Point(int x, int y)) {
 System.out.println(x);
}
```



What improves	What to watch out for
Extract record components during matching.	The record must match the pattern; understand null behavior.

Pause & predict - do not peek

Does Point(int x, int y) create a new Point?

Key point to remember

Record patterns: preview: Extract record components during matching.

Check your reasoning | Java 19

Answer

No. It reads components of an existing Point.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 19. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 20 | Scoped values: incubating

FIRST: what problem are we solving?

Passing request context through many methods can be awkward.

Before this feature: what developers wrote

```
// Often ThreadLocal or explicit parameters
static final ThreadLocal<String> USER =
    new ThreadLocal<>();
```

What is annoying about the old approach?

Passing request context through many methods can be awkward. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Incubator in Java 20; API changed later.
// Learn the concept, not old syntax.
```



What improves	What to watch out for
A controlled way to share context down a call chain.	Not a final Java 20 feature; API evolved in later releases.

Pause & predict - do not peek

Should you memorize Java 20 incubator syntax?

Key point to remember

Scoped values: incubating: A controlled way to share context down a call chain.

Check your reasoning | Java 20

Answer

No. Focus on concept and the API in your target JDK.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 20. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 20 | Virtual threads: second preview

FIRST: what problem are we solving?

Developers tested the virtual-thread model before final release.

Before this feature: what developers wrote

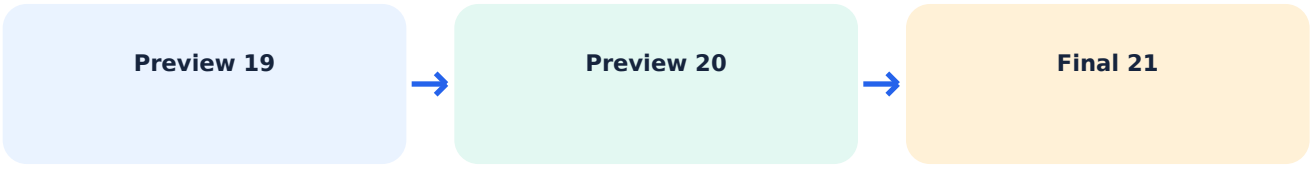
```
// Platform thread per blocking request
new Thread(() -> fetchData()).start();
```

What is annoying about the old approach?

Developers tested the virtual-thread model before final release. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Preview in Java 20; final in Java 21
Thread.startVirtualThread(() -> fetchData());
```



What improves	What to watch out for
Continued testing and refinement before finalization.	Preview features are not guaranteed stable.

Pause & predict - do not peek

Which version finalized virtual threads?

Key point to remember

Virtual threads: second preview: Continued testing and refinement before finalization.

Check your reasoning | Java 20

Answer

Java 21.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 20. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 21 | Virtual threads: final

FIRST: what problem are we solving?

A service handles thousands of requests that spend time waiting for databases.

Before this feature: what developers wrote

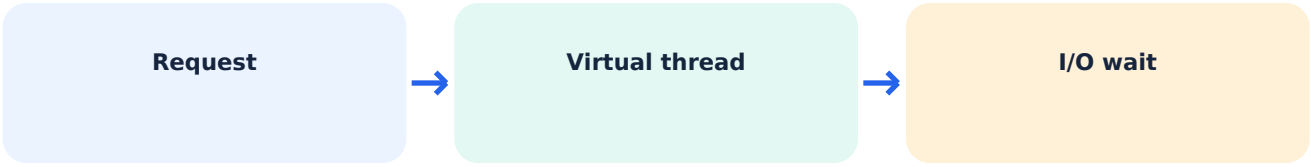
```
ExecutorService pool =
    Executors.newFixedThreadPool(100);
```

What is annoying about the old approach?

A service handles thousands of requests that spend time waiting for databases. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
try (var exec = Executors
    .newVirtualThreadPerTaskExecutor()) {
 exec.submit(() -> fetchData());
}
```



What improves	What to watch out for
Simple thread-per-task style for high I/O concurrency.	Use care with thread-local state and synchronized blocking; not a CPU speed boost.

Pause & predict - do not peek

Should you pool virtual threads like platform threads?

Key point to remember

Virtual threads: final: Simple thread-per-task style for high I/O concurrency.

Check your reasoning | Java 21

Answer

Generally no. Create them per task; limit scarce resources separately.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 21. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 21 | Pattern matching for switch: final

FIRST: what problem are we solving?

Many instanceof checks and casts make branching repetitive.

Before this feature: what developers wrote

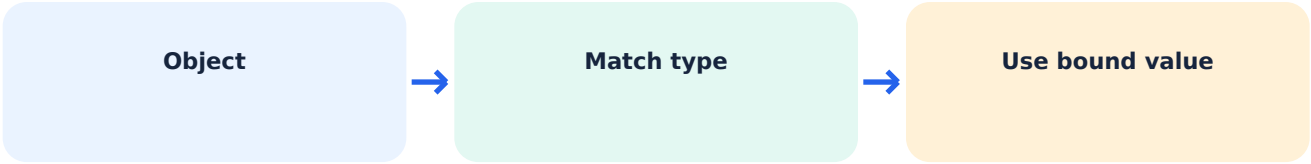
```
if (obj instanceof String) {
 String s = (String)obj;
 System.out.println(s.length());
}
```

What is annoying about the old approach?

Many instanceof checks and casts make branching repetitive. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
String label = switch (obj) {
 case String s -> "text " + s.length();
 case Integer i -> "number " + i;
 default -> "other";
};
```



What improves	What to watch out for
Readable type-based branching with bound variables.	Handle null and exhaustive cases deliberately.

Pause & predict - do not peek

Does default always match null?

Key point to remember

Pattern matching for switch: final: Readable type-based branching with bound variables.

Check your reasoning | Java 21

Answer

No. Null handling has its own rules; use case null when needed.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 21. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 21 | Sequenced collections

FIRST: what problem are we solving?

Getting first/last elements differs across lists, sets and maps.

Before this feature: what developers wrote

```
String first = list.get(0);
String last = list.get(list.size()-1);
```

What is annoying about the old approach?

Getting first/last elements differs across lists, sets and maps. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
String first = list.getFirst();
String last = list.getLast();
var reversed = list.reversed();
```



What improves	What to watch out for
Consistent first/last/reversed operations across ordered collection interfaces.	Not every Set has a defined order; reversed may be a view, not a copy.

Pause & predict - do not peek

Does reversed() necessarily copy elements?

Key point to remember

Sequenced collections: Consistent first/last/reversed operations across ordered collection interfaces.

Check your reasoning | Java 21

Answer

No. It generally provides a reverse-ordered view.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 21. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 21 | Record patterns: final

FIRST: what problem are we solving?

Nested data objects require multiple accessor calls.

Before this feature: what developers wrote

```
if (obj instanceof Point p) {
 int x = p.x();
 int y = p.y();
}
```

What is annoying about the old approach?

Nested data objects require multiple accessor calls. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
if (obj instanceof Point(int x, int y)) {
 System.out.println(x + y);
}
```



What improves	What to watch out for
Directly bind record components in a type check.	Only works with records and valid matching patterns.

Pause & predict - do not peek

Are x and y copies of the whole Point?

Key point to remember

Record patterns: final: Directly bind record components in a type check.

Check your reasoning | Java 21

Answer

No. They are values extracted from its components.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 21. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

Java 21 | Preview features vs final features

FIRST: what problem are we solving?

You see a blog example and wonder if it is safe to use in production.

Before this feature: what developers wrote

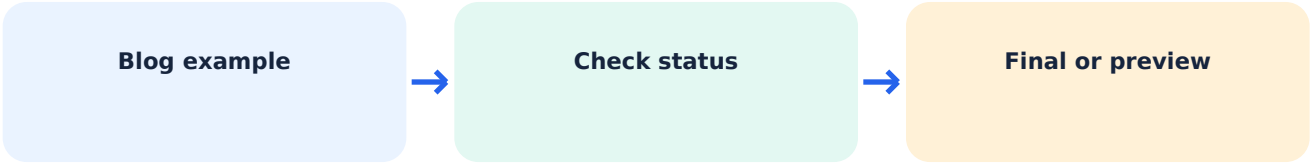
```
// Older posts may show preview APIs.
// Code can change between JDK releases.
```

What is annoying about the old approach?

You see a blog example and wonder if it is safe to use in production. The old approach works, but can be longer, less clear, or less safe for this particular use case.

New approach: write this instead

```
// Java 21 preview: structured concurrency
// Java 21 preview: scoped values
// Java 21 preview: string templates
```



What improves	What to watch out for
Separate stable features from experiments when choosing production APIs.	String templates were preview in Java 21, not finalized there.

Pause & predict - do not peek

Are all Java 21 features production-final?

Key point to remember

Preview features vs final features: Separate stable features from experiments when choosing production APIs.

Check your reasoning | Java 21

Answer

No. Check preview status before using them.

Explain it aloud: Which object changes? Which line runs? Is the result a new container, a view, or a value? If you cannot explain it yet, revisit the diagram and run the snippet.

Mini practice

Change one input value in the example. Predict the output before running it. Then write one sentence describing when you would use this feature in a real application.

Version note

This lesson belongs to Java 21. For preview or incubator features, use the finalized API in your target JDK instead of assuming the early syntax is still supported.

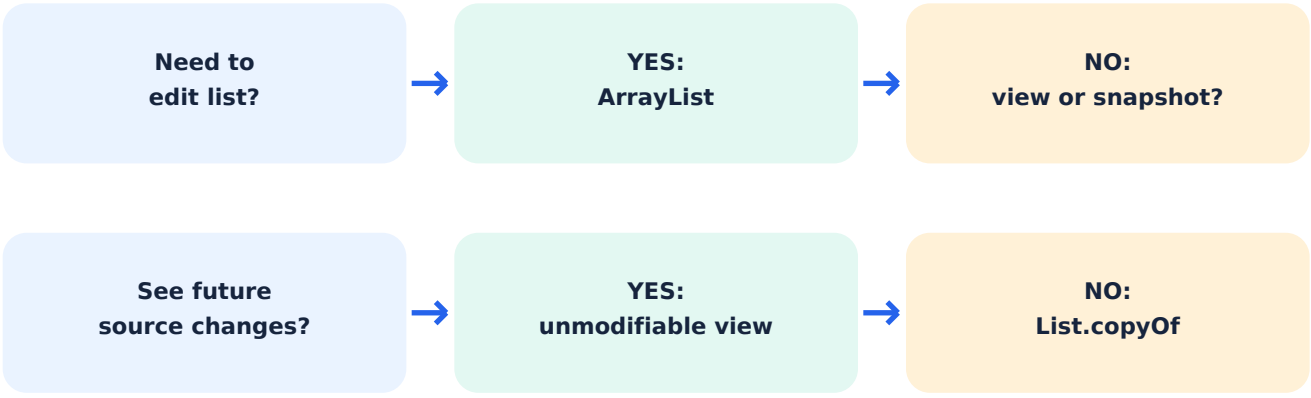
Master cheat sheet: choose the right list

Need	Use	Why
Change elements freely	new ArrayList<>()	Mutable container
Fixed-size view over an array	Arrays.asList(array)	Array and list see replacements
Read-only live view	Collections.unmodifiableList(list)	Source edits remain visible
Mutable separate list	new ArrayList<>(list)	New container, shared elements
Known unmodifiable values	List.of(a, b)	Short and no add/set/remove
Snapshot of existing list	List.copyOf(list)	Source structural changes do not show

Do not confuse these

final prevents reassignment of a variable. Unmodifiable prevents edits through a list reference. Shallow copy means element objects may still be shared. A view reads from another container. A snapshot captures membership at one moment.

Decision flow



Your final challenge

```
List<String> original = new ArrayList<>();
original.add("Java");
List<String> live = Collections.unmodifiableList(original);
List<String> snap = List.copyOf(original);
original.add("Python");
System.out.println(live);
System.out.println(snap);
```

Predict both lines. Answer: [Java, Python] and [Java]. The live view follows the original; the snapshot does not.

Where to go next

Practice with a small payment service: fixed supported methods (List.of), changing transaction history (ArrayList), a read-only live view (unmodifiableList), and a historical snapshot (List.copyOf). Then rebuild its DTOs with records and test concurrency with virtual threads.

Reference: Oracle JDK release notes and language changes at docs.oracle.com; dev.java learning materials. Feature status is described for Java 9 through Java 21.

Expanded lesson 01 | Java 9: List.of and defensive copying

1 | The situation - why should we care?

A service publishes supported payment types. Callers should read them, but should not silently change the allowed types. The old code can expose a mutable list, a fixed-size array view, or a read-only wrapper. Those are three different promises.

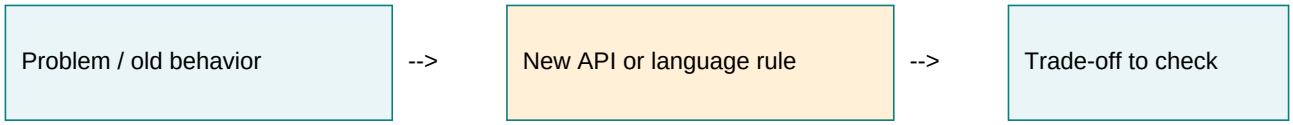
2 | Before: code you might have written

```
List<String> a = new ArrayList<>();
a.add("UPI"); a.add("CARD");
List<String> b = Arrays.asList("UPI", "CARD");
List<String> c = Collections.unmodifiableList(a);
// a.add("CASH") also changes what c displays.
```

3 | After: the new approach

```
List<String> fixed = List.of("UPI", "CARD");
// fixed.set(0, "CASH"); // throws
String[] source = {"UPI", "CARD"};
List<String> snapshot = List.of(source);
source[0] = "CASH"; // snapshot still has UPI
```

4 | Visual mental map



5 | What improved, and what did not?

List.of makes an unmodifiable list without keeping the supplied array as a backing array. It copies element references, not mutable objects. It rejects null.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | List.of and defensive copying

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

What happens if you put a mutable Payment object inside List.of and change payment.method later?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

The object changes; the list still points at that same object.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 02 | Java 9: Set.of and duplicate values

1 | The situation - why should we care?

A registration system has a short set of allowed roles. A set means each value appears at most once. Unlike a list, the order is not a promise you should depend on.

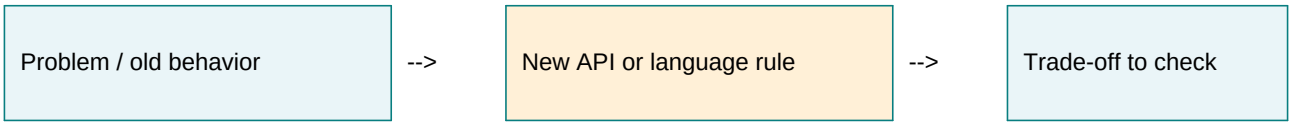
2 | Before: code you might have written

```
Set<String> roles = new HashSet<>();
roles.add("ADMIN"); roles.add("USER");
roles.add("ADMIN"); // silently ignored
```

3 | After: the new approach

```
Set<String> roles = Set.of("ADMIN", "USER");
// Set.of("ADMIN", "ADMIN");
// throws IllegalArgumentException
```

4 | Visual mental map



5 | What improved, and what did not?

Set.of rejects duplicates at creation, rejects null, and has unspecified iteration order. Do not use its order for display.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Set.of and duplicate values

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

What happens with Set.of("A", "A")?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

IllegalArgumentException.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 03 | Java 9: Map.of and Map.ofEntries

1 | The situation - why should we care?

A service maps country codes to labels. A mutable map is convenient while building data, but dangerous when exposed as fixed configuration.

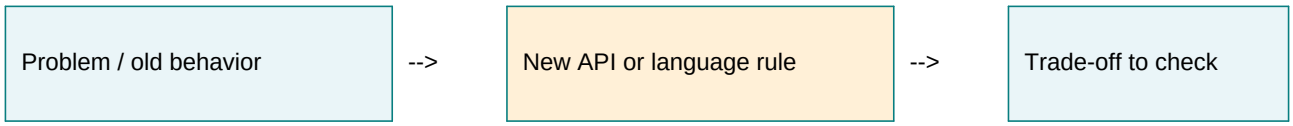
2 | Before: code you might have written

```
Map<String,String> m = new HashMap<>();
m.put("IN", "India");
m.put("US", "United States");
```

3 | After: the new approach

```
Map<String,String> m = Map.of(
    "IN", "India", "US", "United States");
Map<String,Integer> many = Map.ofEntries(
    Map.entry("A", 1), Map.entry("B", 2));
```

4 | Visual mental map



5 | What improved, and what did not?

Map.of supports up to 10 key/value pairs; Map.ofEntries handles more. Duplicate keys and null keys or values are rejected.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Map.of and Map.ofEntries

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

What happens with Map.of("A",1,"A",2)?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

IllegalArgumentException: duplicate key.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 04 | Java 9: Streams: takeWhile, dropWhile, ofNullable

1 | The situation - why should we care?

You process sorted readings and want values only until a limit is crossed. A filter does not stop at the first failure; it examines all items.

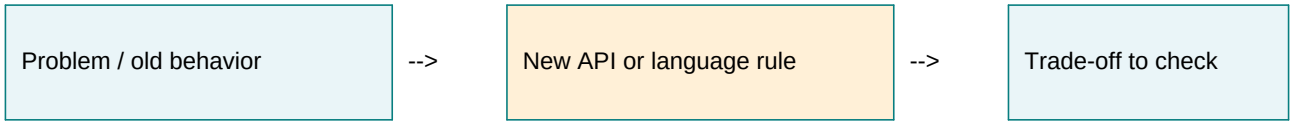
2 | Before: code you might have written

```
List<Integer> n = Arrays.asList(1,2,7,3);
System.out.println(n.stream()
    .filter(x -> x < 5).collect(Collectors.toList()));
// [1, 2, 3]
```

3 | After: the new approach

```
System.out.println(n.stream()
    .takeWhile(x -> x < 5).toList());
// [1, 2]
System.out.println(n.stream()
    .dropWhile(x -> x < 5).toList());
// [7, 3]
Stream.ofNullable(null).count(); // 0
```

4 | Visual mental map



5 | What improved, and what did not?

takeWhile/dropWhile use the beginning of an ordered stream; they are not replacements for filter. Stream.toList itself arrived in Java 16.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Streams: takeWhile, dropWhile, ofNullable

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Why is 3 missing from takeWhile output?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

takeWhile stops when it reaches 7.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 05 | Java 9: Optional and interface improvements

1 | The situation - why should we care?

A lookup might not find a customer. Java 8 Optional helps avoid direct null checks, but some common follow-up actions required extra code.

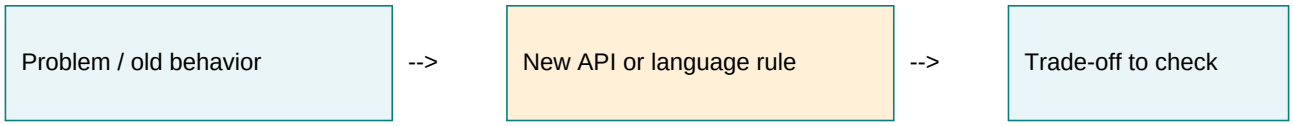
2 | Before: code you might have written

```
Optional<String> result = findName();
if (result.isPresent()) {
    System.out.println(result.get());
} else {
    System.out.println("missing");
}
```

3 | After: the new approach

```
result.ifPresentOrElse(
    System.out::println,
    () -> System.out.println("missing"));
Optional<String> fallback = result.or(() ->
    Optional.of("guest"));
Stream<String> s = result.stream();
```

4 | Visual mental map



5 | What improved, and what did not?

or() expects a Supplier of Optional, unlike orElse(). Optional.stream makes it easier to flatten streams of Optionals. Java 9 also permits private helper methods in interfaces.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Optional and interface improvements

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Does orElse(expensive()) eagerly call expensive()?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

Yes; orElseGet(() -> expensive()) defers it.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 06 | Java 9: JPMS: module-info.java

1 | The situation - why should we care?

In a large application, packages may accidentally depend on internals of other libraries. Modules let a library say what it requires and which packages it makes public.

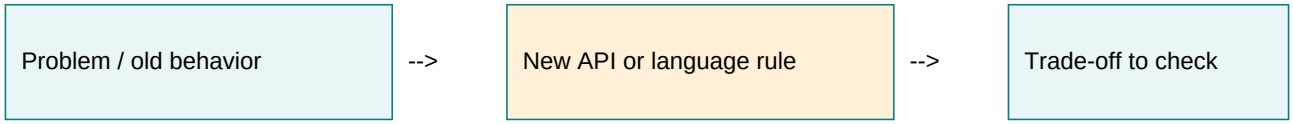
2 | Before: code you might have written

```
// Before modules, classes on the classpath
// may reference any accessible public class
// from another JAR.
```

3 | After: the new approach

```
// module-info.java in billing module
module billing {
    requires java.sql;
    exports com.example.billing.api;
}
// com.example.billing.internal is NOT exported
```

4 | Visual mental map



5 | What improved, and what did not?

Module means a named group of packages with declared dependencies. exports opens compile-time/public access; opens is for reflective access. Classpath applications can still run without modularizing.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | JPMS: module-info.java

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Can another module use a public class in an unexported package?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

Not through normal module access.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 07 | Java 10: var and List.copyOf

1 | The situation - why should we care?

Local variable types can be long and repeated. Also, a caller sometimes needs a read-only snapshot of a list, rather than a live view.

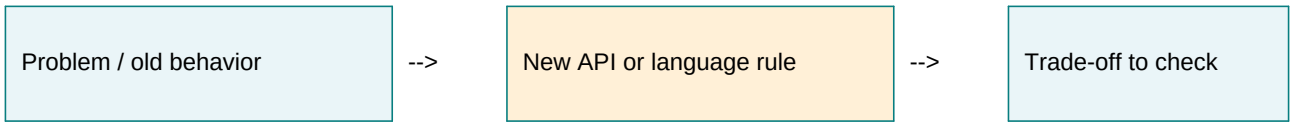
2 | Before: code you might have written

```
Map<String,List<Integer>> scores =
    new HashMap<String,List<Integer>>();
List<String> snap = Collections.unmodifiableList(
    new ArrayList<>(source));
```

3 | After: the new approach

```
var scores = new HashMap<String,List<Integer>>();
List<String> snap = List.copyOf(source);
source.add("new"); // snap membership unchanged
```

4 | Visual mental map



5 | What improved, and what did not?

var is compile-time type inference for local variables, not dynamic typing. List.copyOf rejects null and shares mutable element objects; it may reuse a suitable unmodifiable input.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | var and List.copyOf

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Can you write var value = null;?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No, the compiler cannot infer a type.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 08 | Java 11: String helpers and HTTP Client

1 | The situation - why should we care?

Old code needed trim-based checks and extra work to read HTTP responses. trim and strip differ for Unicode whitespace.

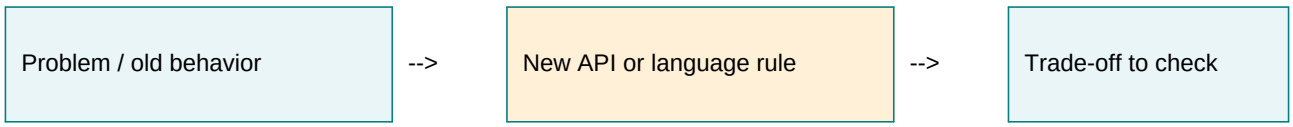
2 | Before: code you might have written

```
boolean blank = text.trim().isEmpty();
// Old HTTP code often used HttpURLConnection.
```

3 | After: the new approach

```
boolean blank = text.isBlank();
String cleaned = text.strip();
String repeated = "ha".repeat(3);
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder(
    URI.create("https://example.org")).build();
```

4 | Visual mental map



5 | What improved, and what did not?

Java 11 standard HTTP Client supports async requests and HTTP/2. strip is Unicode-aware; trim uses older <= U+0020 behavior.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | String helpers and HTTP Client

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Does " ".isBlank() return true?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

Yes.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 09 | Java 12-14: Switch expressions: preview to final

1 | The situation - why should we care?

A switch that calculates a value needs a temporary variable, repeated assignments, and break statements. Missing break may accidentally run the next case.

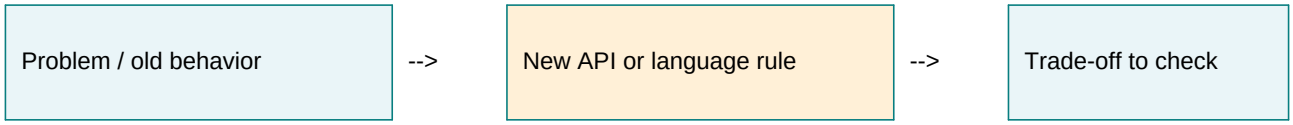
2 | Before: code you might have written

```
String label;
switch (day) {
  case 1: label = "Mon"; break;
  case 2: label = "Tue"; break;
  default: label = "Other";
}
```

3 | After: the new approach

```
String label = switch (day) {
  case 1 -> "Mon";
  case 2 -> "Tue";
  default -> "Other";
};
```

4 | Visual mental map



5 | What improved, and what did not?

Preview in Java 12/13; finalized in Java 14. Switch expressions must produce a result on every path. Arrow cases do not fall through.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Switch expressions: preview to final

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

What happens if you omit default for a non-exhaustive int switch expression?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

Compilation fails because not every input has a result.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 10 | Java 13-15: Text blocks: preview to final

1 | The situation - why should we care?

A JSON request embedded in Java code is hard to read when every quotation mark and line break must be escaped.

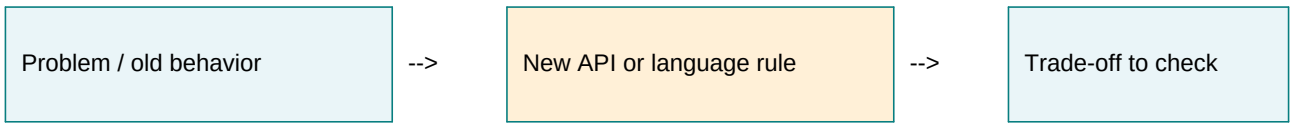
2 | Before: code you might have written

```
String json = "{\n" +
    "  \"name\": \"Asha\"\n" +
    "}";
```

3 | After: the new approach

```
String json = """
    {
      "name": "Asha"
    }
    """;
```

4 | Visual mental map



5 | What improved, and what did not?

Text blocks became final in Java 15. Indentation and trailing newline matter; print the actual string when comparing it with an API contract.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Text blocks: preview to final

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Do text blocks automatically parse JSON?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No, they are still String values.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 11 | Java 14-16: Records: preview to final

1 | The situation - why should we care?

A simple data carrier often needs a constructor, getters, equals, hashCode, and toString. Hand-writing them adds noise and mistakes.

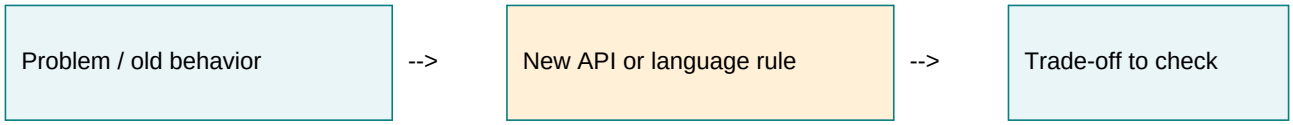
2 | Before: code you might have written

```
final class Point {
  private final int x, y;
  Point(int x, int y) { this.x=x; this.y=y; }
  int x() { return x; }
  int y() { return y; }
}
```

3 | After: the new approach

```
record Point(int x, int y) {}
Point p = new Point(2, 3);
System.out.println(p.x());
System.out.println(p);
```

4 | Visual mental map



5 | What improved, and what did not?

Final in Java 16. Records are shallowly immutable: a record component that is a mutable List can still change unless you defensively copy it.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Records: preview to final

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Does record Box(List<String> xs) {} freeze xs?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No; use a compact constructor and List.copyOf(xs) if needed.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 12 | Java 14-16: Pattern matching for instanceof

1 | The situation - why should we care?

After checking an object type, old code repeats the type in a cast. A cast tells Java to treat the object as that type.

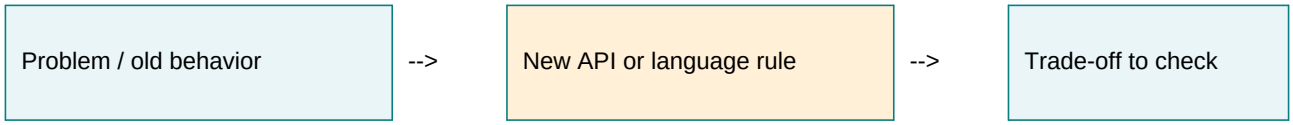
2 | Before: code you might have written

```
if (obj instanceof String) {
  String name = (String) obj;
  System.out.println(name.length());
}
```

3 | After: the new approach

```
if (obj instanceof String name) {
  System.out.println(name.length());
}
```

4 | Visual mental map



5 | What improved, and what did not?

Final in Java 16. The pattern variable exists only where the type check guarantees it is safe. Null never matches a type pattern.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Pattern matching for instanceof

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

What happens when obj is null?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

The instanceof condition is false.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 13 | Java 16: Stream.toList and unmodifiable results

1 | The situation - why should we care?

Collecting a stream to a list used to require a collector. But changing to the shorter method changes mutability expectations.

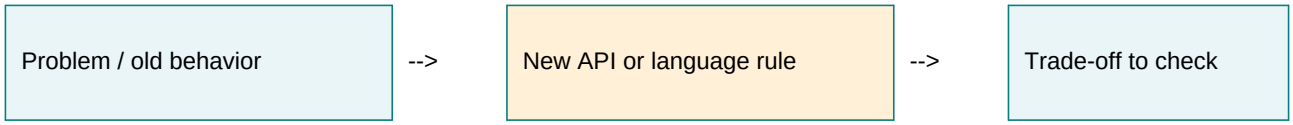
2 | Before: code you might have written

```
List<String> a = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

3 | After: the new approach

```
List<String> b = names.stream()
    .map(String::toUpperCase)
    .toList();
// b.add("X") throws
```

4 | Visual mental map



5 | What improved, and what did not?

Stream.toList() returns an unmodifiable list. Collectors.toList() makes no guarantee about list type or mutability, even though current implementations often return ArrayList.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Stream.toList and unmodifiable results

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Is Stream.toList the same as Collectors.toList?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No: notably, Stream.toList is unmodifiable.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 14 | Java 17: Sealed classes

1 | The situation - why should we care?

A payment model should allow only a known set of implementations. Without a restriction, any other class may extend a public non-final base class.

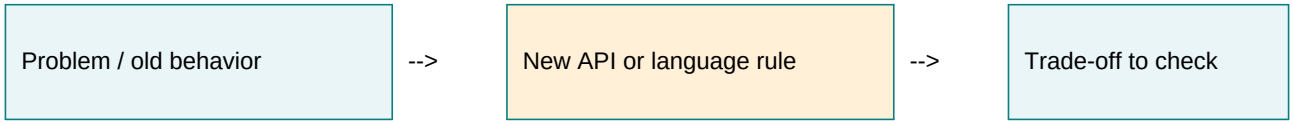
2 | Before: code you might have written

```
interface Payment {}
class Card implements Payment {}
class Cash implements Payment {}
```

3 | After: the new approach

```
sealed interface Payment permits Card, Cash {}
record Card(String last4) implements Payment {}
record Cash() implements Payment {}
```

4 | Visual mental map



5 | What improved, and what did not?

Final in Java 17. Permitted subtypes must follow final, sealed, or non-sealed rules (records are final). This models closed families of types.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Sealed classes

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Can an unrelated class implement Payment?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No, unless it is a permitted subtype.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 15 | Java 18: Simple web server and UTF-8 default

1 | The situation - why should we care?

For small local demos, starting a web server used to need a library or more setup. Text files also behaved differently across machines when the default charset varied.

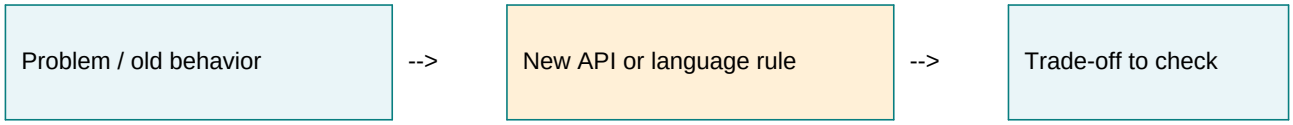
2 | Before: code you might have written

```
// Java 8: common to install a server library
// and explicitly select UTF-8 for portable I/O.
```

3 | After: the new approach

```
// JDK 18: jwebserver command (simple local server)
// JEP 400: UTF-8 became the default charset
// for standard Java APIs, with exceptions.
```

4 | Visual mental map



5 | What improved, and what did not?

The simple web server is for testing and development, not production. Explicit charset choices remain wise for fixed protocols.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Simple web server and UTF-8 default

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Should you use jwebserver as your public production server?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 16 | Java 19-20: Virtual threads previews

1 | The situation - why should we care?

A server handling thousands of requests can spend most time waiting on network or database I/O. A platform thread per request can consume substantial resources.

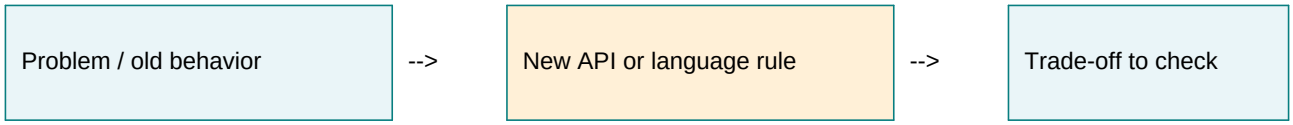
2 | Before: code you might have written

```
ExecutorService pool =
    Executors.newFixedThreadPool(100);
pool.submit(() -> fetchRemoteData());
```

3 | After: the new approach

```
// Java 19/20 preview direction:
// virtual threads are lightweight JVM-managed threads
// Final API in Java 21:
try (var executor = Executors
    .newVirtualThreadPerTaskExecutor()) {
  executor.submit(() -> fetchRemoteData());
}
```

4 | Visual mental map



5 | What improved, and what did not?

Virtual threads help scale blocking I/O, not CPU-heavy loops. Avoid pooling virtual threads as a resource-control mechanism; use semaphores or connection limits instead.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Virtual threads previews

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Will 1000 CPU-heavy virtual threads finish 1000x faster?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No, CPU cores remain the bottleneck.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 17 | Java 21: Virtual threads finalized

1 | The situation - why should we care?

An HTTP handler waits for a slow remote service. We want simple blocking code without requiring a huge pool of heavyweight platform threads.

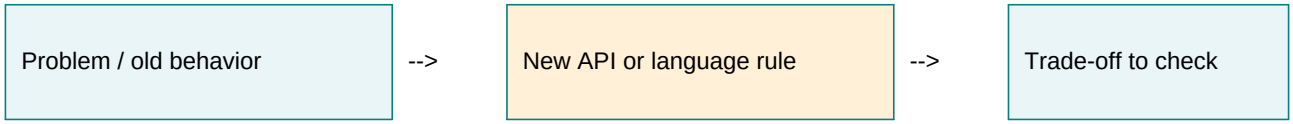
2 | Before: code you might have written

```
ExecutorService pool =
    Executors.newFixedThreadPool(50);
for (int i=0; i<1000; i++)
    pool.submit(() -> blockingCall());
```

3 | After: the new approach

```
try (var executor = Executors
    .newVirtualThreadPerTaskExecutor()) {
  for (int i=0; i<1000; i++)
    executor.submit(() -> blockingCall());
}
```

4 | Visual mental map



5 | What improved, and what did not?

Final in Java 21. ThreadLocal values can be expensive at very large thread counts. In Java 21, some synchronized blocking operations can pin carriers. Test libraries and connection limits.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Virtual threads finalized

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Should you replace every fixed pool with virtual threads?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No. First check if work is I/O-bound and what concurrency limits protect downstream systems.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 18 | Java 21: Record patterns and switch patterns

1 | The situation - why should we care?

Nested data structures often require instanceof checks, casts, and getters. Patterns let the code test and extract matching parts together.

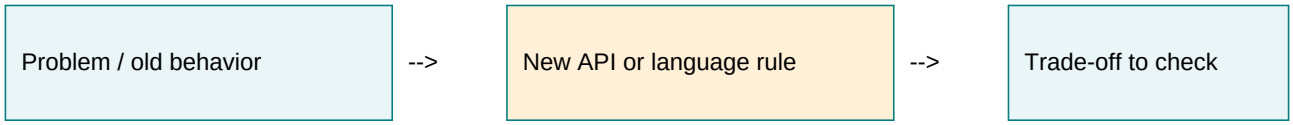
2 | Before: code you might have written

```
if (obj instanceof Point) {
  Point p = (Point) obj;
  System.out.println(p.x());
}
```

3 | After: the new approach

```
record Point(int x, int y) {}
if (obj instanceof Point(int x, int y)) {
  System.out.println(x + y);
}
String result = switch (obj) {
  case String s -> "text: " + s;
  case null -> "missing";
  default -> "other";
};
```

4 | Visual mental map



5 | What improved, and what did not?

Both record patterns and pattern matching for switch became final in Java 21. A switch over a sealed type can often be exhaustive without default.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Record patterns and switch patterns

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Does case String s match null?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No; use case null when needed.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 19 | Java 21: Sequenced collections

1 | The situation - why should we care?

Developers frequently need the first or last item of an ordered collection. Before Java 21, different collection types had different APIs for this.

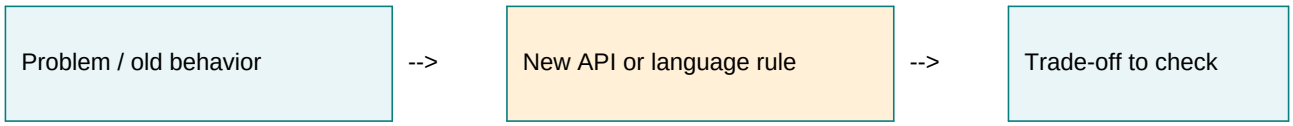
2 | Before: code you might have written

```
List<String> xs = new ArrayList<>(List.of("A","B"));
String first = xs.get(0);
String last = xs.get(xs.size()-1);
```

3 | After: the new approach

```
List<String> xs = new ArrayList<>(List.of("A","B"));
System.out.println(xs.getFirst());
System.out.println(xs.getLast());
System.out.println(xs.reversed());
```

4 | Visual mental map



5 | What improved, and what did not?

SequencedCollection offers first/last/reversed APIs for ordered collections. reversed() returns a reverse-ordered view, not a deep copy. Empty collections throw NoSuchElementException for getFirst/getLast.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Sequenced collections

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

If you change the original list, can reversed() reflect it?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

Yes, it is a view.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 20 | Java 21: Preview features: scoped values and structured concurrency

1 | The situation - why should we care?

A request may start several child tasks. It is hard to track their lifetime and pass request context safely. Java 21 explored APIs to improve these patterns.

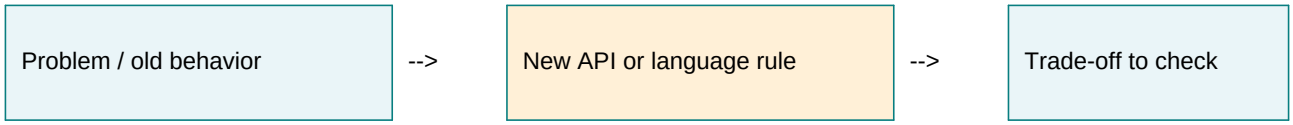
2 | Before: code you might have written

```
// Traditional approach:
// pass context through parameters, ThreadLocal,
// and manually manage multiple Future objects.
```

3 | After: the new approach

```
// Java 21 preview/incubator concepts:
// ScopedValue: scoped read-only context binding
// StructuredTaskScope: group child tasks in a scope
// Do not treat Java 21 preview APIs as stable.
```

4 | Visual mental map



5 | What improved, and what did not?

These were NOT finalized Java 21 features. Their APIs evolved in later releases. Study their goals here, then consult the target JDK docs for actual syntax.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | Preview features: scoped values and structured concurrency

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Can you rely on these APIs in Java 21 without preview/incubator setup?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Expanded lesson 21 | Java 9-21: JVM migration and garbage collection

1 | The situation - why should we care?

An old Java 8 service may compile but fail on a newer JDK because internal APIs, bundled libraries, or startup flags changed. This is different from learning new language syntax.

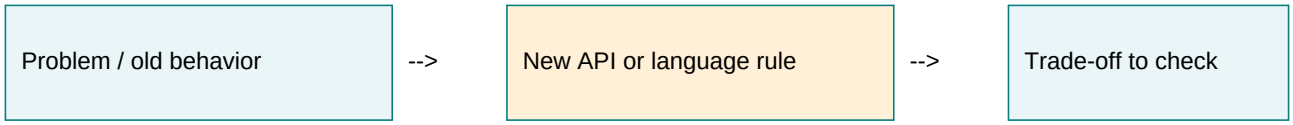
2 | Before: code you might have written

```
// Example Java 8-era dependency:
import javax.xml.bind.JAXBContext;
// JAXB was bundled in older JDKs.
```

3 | After: the new approach

```
// Java 11+: add JAXB as an explicit dependency.
// Test startup flags and reflective libraries.
// Inspect runtime with: java -version
// Inspect class files with: javap -verbose MyClass
```

4 | Visual mental map



5 | What improved, and what did not?

Java 9 made G1 the default GC; Java 11 removed Java EE/CORBA modules; Java 17 strongly encapsulated most JDK internals. Generational ZGC arrived in Java 21 as an option.

Pause: do not read the answer yet. Run the examples, change one input, and predict what happens.

Practice lab | JVM migration and garbage collection

Explain it in your own words

Before writing code, say: What object is created? Who can change it? Is this a copy, a view, or a new value? What would the Java 8 version do?

Predict the result

Does changing -source to 21 automatically fix missing JAXB?

Make the experiment harder

Exercise A: Copy the old code into a small main method. Run it. Change one value and write down the output.

Exercise B: Replace only the relevant lines with the newer code. Explain whether the output or only the readability changed.

Exercise C: Invent one case where the older approach is still appropriate. Do not assume newer always means better.

Answer and explanation

No. Dependency and runtime migration must be handled separately.

Interview-style follow-ups

1. What version introduced this? 2. What was the exact limitation before it? 3. Is there a runtime or compatibility caveat? 4. When would you avoid it?

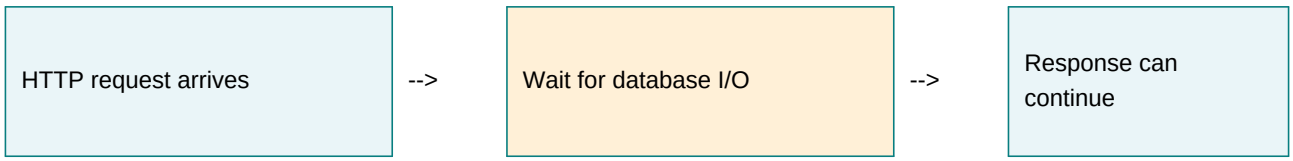
Remember

What to ask	Your check
Behavior	Predict output before running
Ownership	Who can mutate shared objects?
Compatibility	Does the target JDK support this syntax/API?
Decision	Use it only when its semantics match the requirement

Deep dive | Thread scheduling and I/O

First define the terms

A task is work your program wants done. A thread is an execution path that runs code. Blocking means the thread waits for something, such as a database response. A carrier is a platform thread that runs a virtual thread while it is active.



Traditional pool

```
try (var pool = Executors.newFixedThreadPool(10)) {
  for (int i=0; i<100; i++)
    pool.submit(() -> callDatabase());
}
```

Virtual thread per task

```
try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
  for (int i=0; i<100; i++)
    pool.submit(() -> callDatabase());
}
```

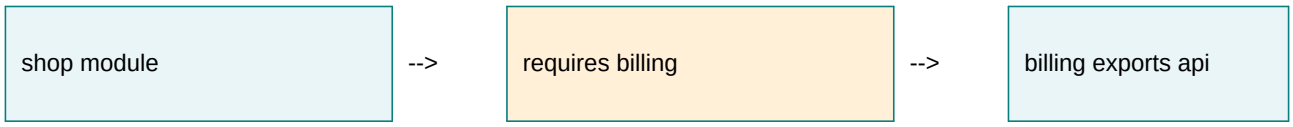
Neither approach increases the number of database connections by magic. A connection pool may allow only 20 connections. Excess tasks still wait.

Experiment

Time 100 tasks that each sleep 200 ms, then compare with 100 tasks doing CPU-intensive arithmetic. Why are the results different?

Deep dive | Modules: two boxes, one doorway

A package is a group of related classes. A module is a named group of packages with an explicit doorway: exports. Public does not automatically mean accessible across module boundaries.




```
// billing/module-info.java
module billing { exports demo.billing.api; }
// shop/module-info.java
module shop { requires billing; }
// shop may use demo.billing.api.Invoice
// shop may NOT use demo.billing.internal.Secret
```

Try this

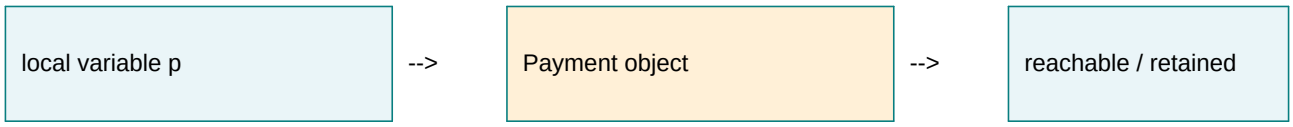
Move Invoice from api to internal and predict the compilation error. Then add an exports statement and try again.

Why this matters

Large systems can accidentally rely on internal classes. Modules make such boundaries explicit, but they also require care with reflection-heavy frameworks.

Deep dive | Heap, references, and GC

A reference is a value that lets Java find an object. Two variables can hold references to one object. Garbage collection considers whether an object is reachable from roots such as live local variables and static fields.




```
Payment p = new Payment("UPI");
Payment q = p; // same object
p = null;    // q still keeps object reachable
q = null;    // may now be collectible
```

Setting q to null does not immediately destroy the object. The garbage collector decides when to reclaim eligible memory.

G1 vs ZGC in plain language

Garbage collectors manage heap memory. G1 is a general-purpose collector and became the default in Java 9. ZGC targets low pauses; Java 21 added a generational mode. The right collector depends on measured application behavior, not a universal ranking.

Practice

Draw a reference diagram with three variables and two objects. Cross out references one at a time. Identify when each object stops being reachable.

Collection decision map | Keep this beside your IDE

You need	Use	Why / warning
Add or remove elements	new ArrayList<>()	Mutable list
Fixed-size array-backed view	Arrays.asList(array)	set works; add does not
Read-only live view	Collections.unmodifiableList(source)	Source changes are visible
Mutable independent list	new ArrayList<>(source)	Shallow copy
Read-only snapshot, Java 8	unmodifiableList(new ArrayList<>(source))	Shallow snapshot
Known fixed values, Java 9	List.of("A", "B")	No null; unmodifiable
Snapshot of existing list, Java 10	List.copyOf(source)	No null; unmodifiable
Stream result, Java 16	stream.toList()	Unmodifiable list

Three tricky questions

Q1. Does final List<Payment> make Payment immutable? No. Q2. Does List.of(payment) copy the Payment object? No. Q3. Can a Collections.unmodifiableList view change without calling add on the view? Yes, if the backing list changes.